❖ HTML CSS

Basic HTML and CSS Questions

1. What is HTML?

Answer:

HTML stands for **HyperText Markup Language**. It is used to **structure content** on the web using a system of tags like , <h1>, <a>, , etc.

2. What are tags in HTML?

Answer:

Tags are HTML elements used to define how content should appear on the webpage. Most tags have an opening (<tag>) and a closing (</tag>) form.

Example: This is a paragraph.

3. What is the difference between and ?

Answer:

- is a **block-level** element: takes full width and starts on a new line.
- is an **inline** element: used to style small parts of text inline without breaking lines.

4. What is the purpose of the <!DOCTYPE html> declaration?

Answer:

It defines the **document type** and version of HTML. It tells the browser to render the page in **standard mode**, not quirks mode.

5. What is semantic HTML?

Answer:

Semantic HTML uses **meaningful tags** like <article>, <section>, <nav>, etc., which improve **SEO**, **accessibility**, and **code readability**.

6. What is the difference between id and class in HTML?

Answer:

- id: Unique identifier, used once per page. Syntax: id="main"
- class: Can be used multiple times. Syntax: class="container"

7. What are void (self-closing) elements in HTML?

Answer:

Elements that don’t need a closing tag.

Examples: <img>,
, <input>, <hr>

8 What is the difference between <script> and <link>?

Answer:

- <script> is used to include **JavaScript** files.
- <link> is used to include **CSS** files or other resources.

10. What is the use of the alt attribute in the <img> tag?

Answer:

The alt attribute provides **alternative text** for an image if it fails to load. It also helps screen readers describe images to visually impaired users.

12. What is the purpose of the <meta> tag in HTML?

Answer:

The <meta> tag provides **metadata** about the HTML document, like charset, author, description, and keywords for SEO.

<<<<<<<<<<<<<<CSS question >>>>>>>>>>>>>>>

1. Difference between `display: none`, `visibility: hidden`, and `opacity: 0`

Property	Hidden from View	Takes Up Space	Affects Layout	Can Be Clicked	Best Use Case
display: none	Yes	No	Yes	No	Completely remove from layout
visibility: hidden	Yes	Yes	No	No	Hide but keep structure
opacity: 0	Yes	Yes	No	Yes	Hide but allow interaction

2. CSS positions?

Position	Description	Takes Space	Relative To
static	Default. Element stays in normal document flow.	Yes	Normal flow
relative	Moves relative to its original position.	Yes	Its own original position
absolute	Removed from flow, positioned with top/left etc.	No	Nearest positioned ancestor
fixed	Stuck to viewport, doesn't move on scroll.	No	Browser window
sticky	Acts like relative until scrolls to threshold, then sticks	Yes	Nearest scrollable container

3. Specificity in CSS

Inline styles = 1000 points

ID selectors = 100 points

Class, attribute, pseudo= 10 points

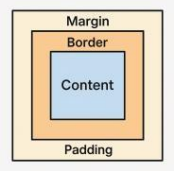
Element, tag, pseudo-element= 1 point

4. Flex box and grid

Feature	Flexbox	Grid
Layout	One-dimensional (row or column)	Two-dimensional (rows and columns)
Direction	One direction at a time	Both directions at once
Use Case	Navbars, cards, toolbars	Full-page layouts, templates
Syntax	Simple	More complex but powerful Nesting
	Needs nesting for 2D	No nesting needed Alignment
	Great in one axis	Great in both axes
Responsiveness	Easy with wrapping	Needs auto-fit or media queries

5. What is the CSS Box Model?

The **CSS Box Model** is a core concept that explains how every HTML element is structured and spaced on a web page. Each element is essentially a rectangular box, made up of the following parts **from the inside out**:



6. What are pseudo-classes and pseudo-elements?

- **Pseudo-classes:** Target element state (e.g., `:hover`, `:focus`)
- **Pseudo-elements:** Style parts of elements (e.g., `::before`, `::after`)

7. What is z-index in CSS?

Answer:
It controls **stacking order**. Higher `z-index` values appear in front of lower ones.

8. Difference between `em`, `rem`, `px`, `%`?

- `px`: Fixed size
- `em`: Relative to parent font size
- `rem`: Relative to root (`html`) font size
- `%`: Relative to parent size

9. What are the types of CSS?

Answer:

- **Inline CSS:** Applied directly in the element using the `style` attribute.
- **Internal CSS:** Defined in the `<style>` tag inside `<head>`.
- **External CSS:** Linked using a `.css` file with `<link>`.

❖ JAVASCRIPT

Async and defer attribute in script tag ?

In HTML, the `async` and `defer` attributes are used in the `<script>` tag to control **how and when JavaScript files are loaded and executed**. These attributes are especially important for optimizing page load performance.

`defer` Attribute:

- **Usage**: The `defer` attribute also downloads the script asynchronously, but ensures the script is executed only after the HTML document has been fully parsed.

`async` Attribute:

- **Usage**: The `async` attribute tells the browser to download the script asynchronously while the HTML document is still being parsed. Once the script is fully downloaded, it is executed immediately, even if the HTML parsing is not complete.

Currying

Currying in JavaScript is a technique in functional programming where a function is transformed into a series of functions, each with a single argument.

Instead of taking all arguments at once, a curried function takes the first argument and returns a new function that takes the second argument, and so on, until all arguments are provided.



JSON.stringify()

Purpose: Converts a JavaScript object or value into a JSON string.

const obj = { name: "Babu", age: 23 };
const jsonString = JSON.stringify(obj);
console.log(jsonString); // Output: '{"name":"Babu","age":23}'

JSON.parse()

Purpose: Converts a JSON string back into a JavaScript object.

```
const jsonString = '{"name":"Babu","age":23}';
const obj = JSON.parse(jsonString);
console.log(obj.name); // Output: Babu
```

JavaScript Variable Declarations: var, let, const

var is function-scoped, while let and const are block-scoped.

1. Global Scope

Variables declared outside of any function or block are in the global scope.

These variables are accessible anywhere in the code (inside functions or

blocks).

Local Scope

Variables declared inside a function are in the local scope.

These variables are accessible only within the function in which they are declared.

1. var

Scope: Function-scoped.

Hoisting: Hoisted and initialized as

undefined. Re-declaration: Allowed.

Re-assignment: Allowed.

Drawbacks: Can lead to unexpected bugs due to lack of block scoping.

Example:

2. let

Scope: Block-scoped.

Hoisting: Hoisted but not initialized (temporal dead

zone). Re-declaration: Not allowed in the same scope.

Re-assignment: Allowed.

Example:

3. const

Scope: Block-scoped.

Hoisting: Hoisted but not initialized (temporal dead

zone). Re-declaration: Not allowed in the same scope.

Re-assignment: Not allowed (constant).

Mutability: Objects and arrays are mutable, but their reference cannot be changed.

```
function testScopes() {

    // `var` is function-scoped, so it's accessible throughout the

    function. if (true) {

        var a = 10; // `var` is function-scoped (accessible anywhere in the function)

        let b = 20; // `let` is block-scoped (only accessible inside this `if` block)

const c = 30; // `const` is block-scoped (only accessible inside this `if` block)

    }

console.log(a); // 10 (accessible anywhere in the function)

    // console.log(b); // Error: b is not defined (b is block-scoped)

    // console.log(c); // Error: c is not defined (c is block-scoped)
```

```
}

testScopes();
```

what is lexical scoping?

Lexical scoping allows inner functions to access variables from their parent scope.

Lexical Scope

JavaScript uses lexical scoping (also known as static scoping), which means that a function's scope is determined by where it is declared, not where it is executed.

Functions can access variables from their parent scope.

```
function outerFunction() {

    let outerVar = "I am from outer scope";

    function innerFunction() {

        console.log(outerVar); // Accessible because of lexical scoping

    }

    innerFunction();

}
outerFunction(); // Output: I am from outer scope
```

The Rest operator is used to collect all remaining elements into a single variable.

It is often used in function parameters or destructuring to gather elements into an array or object

Spread Operator: Expands elements from an iterable (used for merging and copying arrays/objects).

```
const arr1 = [1, 2];

const arr2 = [3, 4];

const merged = [...arr1, ...arr2];
```

console.log(merged); // [1, 2, 3, 4]

When to Use null and undefined:

Use null: When you want to explicitly indicate the absence of any object value or if you're resetting a value.

Example: let person = null; // No person exists at the moment

Use undefined: When a variable is declared but not assigned any value or when you expect missing arguments in a function.

Example: let age; // Age is declared, but not assigned

What is the difference between synchronous and asynchronous execution in JavaScript?

Synchronous: Operations are executed one after the other, blocking the execution of further code until the current task completes.

Asynchronous: Operations are executed in the background, allowing other code to run while waiting for the operation to complete (using setTimeout, Promises, etc.).

event loop ?

The Event Loop keeps checking the call stack and the callback queue.If the call stack is empty, it takes the first function in the callback queue and pushes it onto the call stack for execution.

basically event loop working wetween callback queue and call stack,

what is call stack ?

"The call stack is a data structure in JavaScript that tracks the execution of function calls in your program.

It operates on a Last In, First Out (LIFO) principle, meaning the last function added to the stack is the first to be executed and removed."

What is this?

this refers to the execution context in JavaScript, which changes based on how a function is

called.

In the global execution context (outside of any function), this refers to the global object.

1. When used inside a regular function, this refers to the global object (non-strict mode) or undefined (strict mode).
2. When a function is called as a method of an object, this refers to the object that is called the method.
3. Arrow functions do not have their own this. Instead, they inherit this from the enclosing execution context (lexical scoping).
4.
5.

javascript execution contexte ?

javascript is a sigle threade

1. JS creates Global excution context => alocate to "this" keyword

2. Next it creates memory phase

2. Memory phase - In this phase,the variables are set to undefined

until the execution phase(next phase) and the functions are set to their definitions.

3. Next it creates the execution phase

3. Execution phase - In this phase,the variables are initialsed to given values and

when the function s are called,it creates a memory phase and execution phase for the function

just like the main one.

Returned value is passed to the global execution context

call(),apply() and bind() ?

Function Borrowing: You can borrow functions from one object and apply them to another object without rewriting the function.

where reusing a method across multiple objects dynamically.

When using a function as a callback and needing to set this explicitly.

* apply take argument in array formate and call take argument as single single. and apply and call both function invoke immediately

* bind is similar to call but create new funciton and it store in any variable for call later.

example:

```
const person1 = {
    firstName: "John",
    lastName: "Doe",
    fullName: function () {
        return `${this.firstName} ${this.lastName}`;
    }
};

const person2 = {
    firstName: "Jane",
    lastName:
    "Smith"
};
console.log(person1.fullName.call(person2)); // Output: "Jane Smith"
```

What is a Closure in JavaScript?

A closure is a function that "remembers" the variables from its lexical scope even after the outer function has finished executing.

In simpler terms, a closure allows an inner function to access variables from its outer function,

even after the outer function has returned.

*what is hoisting in js?

Hoisting is JavaScript's default behavior of moving declarations (variables, functions, or classes)

to the top of their containing scope during the compilation phase.

This means you can use variables and functions before declaring them in your code.

Concept	Analogy
Promises	"Order pizza, then watch TV until the pizza arrives."
Async/Await	"Stand in line and wait for the pizza. Do nothing else while waiting."

1. Template Literals:

In JavaScript, backticks are used to create template literals, which are an advanced way to handle

strings. They offer several features that make string handling more flexible and readable.

Template literals (backticks) are a powerful feature in JavaScript that improves string manipulation by supporting interpolation,multi-line strings, and expression evaluation.

What is a Callback in JavaScript?

A callback in JavaScript is a function that is passed as an argument to another function and it executed after the completion of that function.

Callbacks are commonly used to handle asynchronous operations like reading files, making API calls, or handling events.

example:

```
function fetchData(callback) {
```

```
    console.log("Fetching data...");
    setTimeout(() => {
        console.log("Data fetched successfully.");
        callback(); // Call the callback function after fetching the data
    }, 2000);
}

function processData() {
    console.log("Processing the data...");
}

fetchData(processData);
```

Callback Hell

When callbacks are nested within other callbacks, the code becomes hard to read and maintain. This is known as callback hell.

```
setTimeout(() => {
    console.log("Step 1 completed.");
    setTimeout(() => {
        console.log("Step 2 completed.");
        setTimeout(() => {
            console.log("Step 3 completed.");
        }, 1000);
    }, 1000);
}, 1000);
```

what is prototype in JavaScript?

A prototype in JavaScript is an object from which other objects inherit properties and methods. Every object has an internal __proto__ **property that links it to a prototype, forming a prototype chain. We use** prototype **to define shared methods for constructor functions, improving memory efficiency. This enables prototype inheritance, allowing objects to inherit functionality. Prototypes are a key feature of JavaScript's object-oriented programming model.**

OR

What is a Prototype?

Every object in JavaScript has a hidden link (`__proto__`) to another object called its **prototype**. This prototype contains shared properties and methods.

🟢⚫ **Example:**

Imagine a **blueprint for a car**. Instead of giving each car its own copy of functions (like `start()` or `drive()`), all cars **share** these functions from the blueprint.

1️⃣ Promise ? (eventual "Promise guarantees that a value will be available in the future.")

A Promise is a JavaScript object that represents the eventual completion or failure of an asynchronous operation. It provides a reliable way to handle future values, avoid callback hell, and manage asynchronous code in a readable and structured manner.

Example:

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data loaded successfully");
  }, 2000);
});

promise.then(result => {
  console.log(result);
});
```

2️⃣ Promise States

A promise can be in one of three states: pending, fulfilled, or rejected. It starts in the pending state and moves to fulfilled or rejected only once. After settlement, the state and result never change.

1. **Pending** – Initial state (operation is ongoing).
2. **Resolved (Fulfilled)** – Operation completed successfully.
3. **Rejected** – Operation failed.

Example:

```
const p = new Promise((resolve, reject) => {
  resolve("Success");
  reject("Error"); // ignored
```

```
});

p.then(console.log);
```

---

3. then()

Theory:

The then() method registers a callback that executes when a promise is fulfilled. It receives the resolved value and always returns a new promise, enabling promise chaining and sequential asynchronous operations.

Example:

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(users => {
    users.forEach(user => console.log(user.name));
  });
```

---

4. catch()

Theory:

The catch() method handles promise rejections or runtime errors occurring in any previous then() block. It provides centralized and clean error handling in promise chains.

Example:

```
fetch("wrong-url")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(error => {
    console.log("Error occurred:", error.message);
  });
```

---

5. finally()

Theory:

The finally() method runs after a promise settles, regardless of fulfillment or rejection. It is mainly used for cleanup tasks like stopping loaders or closing resources and does not modify the promise result.

Example:

```
fetch("https://api.example.com/data")
  .then(res => res.json())
  .catch(err => console.log(err))
  .finally(() => {
    console.log("Request completed");
  });
```

---

6. Promise Chaining

Theory:

Promise chaining allows multiple asynchronous tasks to execute sequentially by returning a promise from each then() block. This approach improves readability, avoids nested callbacks, and ensures proper flow control.

Example:

```
getUser()
  .then(user => getOrders(user.id))
```

```
  .then(orders => processOrders(orders))
  .then(result => console.log("Done:", result))
  .catch(err => console.log(err));
```

7️⃣ Promise.all()

Theory:

Promise.all() executes multiple promises in parallel and resolves only when all promises are fulfilled. If any promise rejects, the entire operation fails immediately with that error.

Example:

```
Promise.all([
  fetch("https://jsonplaceholder.typicode.com/users").then(res => res.json()),
  fetch("https://jsonplaceholder.typicode.com/posts").then(res => res.json()),
  fetch("https://jsonplaceholder.typicode.com/comments").then(res => res.json()),
  fetch("https://jsonplaceholder.typicode.com/albums").then(res => res.json()),
  fetch("https://jsonplaceholder.typicode.com/photos").then(res => res.json())
])
.then(([users, posts, comments, albums, photos]) => {
  console.log(users, posts, comments, albums, photos);
})
.catch(err => console.log(err));
```

Example 2

```
function loadAllData() {
  Promise.all([
    fetch("https://jsonplaceholder.typicode.com/users").then(res => res.json()),
    fetch("https://jsonplaceholder.typicode.com/posts").then(res => res.json()),
    fetch("https://jsonplaceholder.typicode.com/comments").then(res => res.json()),
    fetch("https://jsonplaceholder.typicode.com/albums").then(res => res.json()),
    fetch("https://jsonplaceholder.typicode.com/photos").then(res => res.json())
  ])
  .then(([users, posts, comments, albums, photos]) => {
    console.log(users, posts, comments, albums, photos);
  })
  .catch(err => console.log(err));
}
```

8️⃣ Promise.allSettled()

Theory:

Promise.allSettled() waits for all promises to settle and returns their individual status and results, making it useful when success and failure must both be handled.

Example:

```
Promise.allSettled([
  Promise.resolve("OK"),
  Promise.reject("Failed")
]).then(results => {
  console.log(results);
});
```

9️⃣ Promise.race()

Theory:
Promise.race() returns the result of the first promise that settles, whether fulfilled or rejected. It is commonly used for timeouts or selecting the fastest response.

Example:

```
Promise.race([
  fetch("/fast-api"),
  new Promise((_, reject) =>
    setTimeout(() => reject("Timeout"), 2000)
  )
]).then(console.log).catch(console.log);
```

🔟 Promise.any()

Theory:
Promise.any() resolves with the first fulfilled promise and ignores rejected ones. It only rejects if all promises fail, returning an AggregateError.

Example:

```
Promise.any([
  Promise.reject("Server down"),
  fetch("/backup-api").then(res => res.json())
]).then(console.log);
```

1️⃣1️⃣ Async/Await with Promise

Async/await is syntactic sugar over promises that makes asynchronous code appear synchronous. It improves readability and simplifies error handling while still relying entirely on promise behavior.

Example:

```
async function loadData() {
  try {
    const res = await fetch("/data");
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log(err);
  }
}
```

1️⃣2️⃣ Promise and Event Loop

Theory:
Promise callbacks are placed in the microtask queue, which is executed before the macrotask queue. This ensures promise handlers run before timers like setTimeout.

Example:

```
console.log("Start");

setTimeout(() => console.log("Timeout"), 0);
```

Promise.resolve().then(() => console.log("Promise"));

console.log("End");

Different between callback, promises and async await ?

Line 1:

“Callback hell occurs when we have many nested callbacks for asynchronous operations, which makes the code very messy, difficult to read, and hard to maintain, especially when handling multiple tasks one after another.”

Line 2:

“Promises solve this problem by allowing chaining, which flattens the code structure, and async/await makes asynchronous code look synchronous, improving readability, simplifying error handling, and making debugging much easier.”

Line 1 – Callback Hell:

“Callback hell happens when we nest many asynchronous callbacks, which makes the code messy, hard to read, and difficult to maintain, especially when handling multiple tasks sequentially.”

Line 2 – Why Promises instead of Callbacks:

“We use promises instead of callbacks because promises flatten the code structure through chaining, avoid deep nesting, and provide a consistent way to handle errors with .catch(), making asynchronous code more manageable and readable.”

Line 3 – Why Async/Await instead of Promises:

“Async/await is used on top of promises because it makes asynchronous code look synchronous, further improving readability, simplifying error handling using try/catch, and making debugging much easier, especially for complex workflows with multiple async calls.”

Callback example: Promises Example: async await Example:



```
function getUser(id, callback) {
  setTimeout(() => {
    console.log("User fetched");
    callback({ id: id, name: "Papa" });
  }, 1000);
}

function getPosts(userId, callback) {
  setTimeout(() => {
    console.log("Posts fetched");
    callback([{ id: 1, title: "Post 1" }]);
  }, 1000);
}

function getComments(postId, callback) {
  setTimeout(() => {
    console.log("Comments fetched");
    callback(["Comment 1", "Comment 2"]);
  }, 1000);
}

// Nested callbacks → callback hell
getUser(1, user => {
  getPosts(user.id, posts => {
    getComments(posts[0].id, comments => {
      console.log("Comments:", comments);
    });
  });
});
```

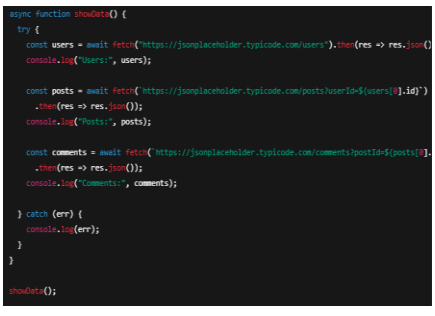


```
function getUser(id) {
  return new Promise(resolve => {
    setTimeout(() => {
      console.log("User fetched");
      resolve({ id: id, name: "Papa" });
    }, 1000);
  });
}

function getPosts(userId) {
  return new Promise(resolve => {
    setTimeout(() => {
      console.log("Posts fetched");
      resolve([{ id: 1, title: "Post 1" }]);
    }, 1000);
  });
}

function getComments(postId) {
  return new Promise(resolve => {
    setTimeout(() => {
      console.log("Comments fetched");
      resolve(["Comment 1", "Comment 2"]);
    }, 1000);
  });
}

// Promise chaining
getUser(1)
  .then(user => getPosts(user.id))
  .then(posts => getComments(posts[0].id))
  .then(comments => console.log("Comments:", comments))
  .catch(err => console.log(err));
```



```
async function showData() {
  try {
    const users = await fetch("https://jsonplaceholder.typicode.com/users").then(res => res.json());
    console.log("Users:", users);

    const posts = await fetch(`https://jsonplaceholder.typicode.com/posts?userId=${users[0].id}`)
      .then(res => res.json());
    console.log("Posts:", posts);

    const comments = await fetch(`https://jsonplaceholder.typicode.com/comments?postId=${posts[0].
      .then(res => res.json());
    console.log("Comments:", comments);

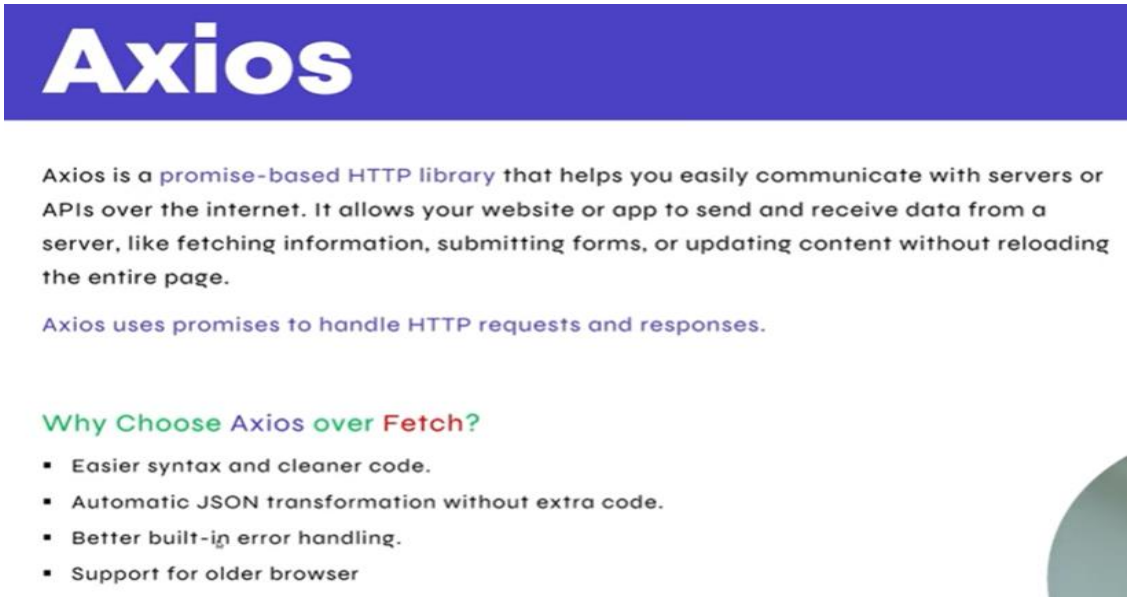
  } catch (err) {
    console.log(err);
  }
}

showData();
```

What is Async/Await in javaScript?

async/await is a modern way to handle asynchronous operations in JavaScript. It makes code more

readable and avoids callback hell. await pauses execution until a Promise resolves, and try/catch is used for error handling. We can also use Promise.all() for running multiple await calls in parallel, improving performance.



Example of Axios:

```
axios.get("https://jsonplaceholder.typicode.com/users")
  .then(res => console.log(res.data)) // JSON parsed automatically
  .catch(err => console.log(err));
```

```
const fetchData = async () => {
  try {
  const data = await axios.get("https://jsonplaceholder.typicode.com/users", { timeout: 500 });  // if network slow so it automatically cancle request
   setData(data.data);
  } catch (err) {
   console.log("API failed or timed out:", err.message);
     }
};
fetchData();
```

```
useEffect(() => {
  const controller = new AbortController();

  axios.get(`https://jsonplaceholder.typicode.com/users`, { signal: controller.signal })
    .then(res => setData(res.data))
    .catch(err => {
      if (err.name === "CanceledError") console.log("Request canceled");
      else console.log(err);
    });

  return () => controller.abort(); // cancel request on component unmount
}, [searchTerm]);
```

- If `searchTerm` changes, old request is canceled automatically

Fetch vs axios

.Difference between Fetch and Axios

Theory:

- Fetch is **native**, Axios is **library**
- Axios automatically parses JSON, handles HTTP errors, supports timeout, interceptors, and works in Node.js.
- Fetch needs manual .json() parsing, error handling for HTTP status, and doesn't support interceptors natively.

Why use Axios instead of Fetch?

Theory:

- Auto JSON parsing
- Handles HTTP errors
- Timeout support
- Interceptors
- Cancel requests
- Works in Node.js

Feature	Fetch	Axios
Type	Browser native API	Library (npm package)
Syntax	Native JS promise-based	Promise-based, shorter syntax
Default JSON	Must call `res.json()`	JSON parsing automatic
Error Handling	Only network errors trigger `.catch()`	Network + status errors trigger `.catch()`
Browser Support	Modern browsers only	Works in older browsers + Node.js
Request Config	Slightly verbose for headers, body	Easy config for headers, params, body
Cancel Request	Not built-in	Built-in `CancelToken`

Arrow vs normal function

: *What is an IIFE***Immediately Invoked Function Expression** *, and why is it used?*

An IIFE (Immediately Invoked Function Expression) is a function that executes immediately after being defined. It helps to avoid polluting the global scope and is commonly used in module patterns."

Definition

WebSocket is a protocol that provides full-duplex, real-time communication between client and server over a single, persistent TCP connection.

- **Unlike HTTP, the connection stays open, allowing both sides to send and receive data anytime.**

2️⃣ Key Features

- **Bi-directional communication 🔁**
- **Low latency (instant updates) ⚡**
- **Persistent connection → no need for repeated HTTP requests**
- **Ideal for real-time apps: chat, live notifications, games, stock tickers**

3️⃣ How it works (simple)

1. **Client sends HTTP request asking to upgrade to WebSocket**
2. **Server accepts → connection established**
3. **Both client & server can send messages anytime without closing connection**

What is the difference between == and === in JavaScript?

"== is a loose equality operator that compares values but allows type conversion (coercion), while === is a strict equality operator that checks both value and type without conversion."

Example

console.log(5 == "5"); // true (Type conversion

happens) console.log(5 === "5"); // false (Type is different)

.... — Array method —

What is array? An array in JavaScript is a type of global object used to store data.

Arrays can store multiple values in a single variable, which can condense and

organize our code.

Basic Terminologies of JavaScript Array

● Array: A data structure in JavaScript that allows you to store multiple values in a single variable.

● Array Element: Each value within an array is called an element. Elements are accessed by their index.

● Array Index: A numeric representation that indicates the position of an element in the array. JavaScript arrays are zero-indexed, meaning the first element is at

index

● Array Length: The number of elements in an array. It can be retrieved using the length property.

Creating an Array using Array Literal let arrayName = [value1, value2, ...];

Creating an Array using new Keyword (Array Constructor) let arrayName = new Array();

let courses = ["HTML", "CSS", "JavaScript", "React"];

Let last=courses[courses.length - 1]; access last element in array

let firstItem = courses[0]; access first element in array

courses[1]= "Bootstrap"; courses = ["HTML", "Bootstrap",

"JavaScript", "React"];

What is Higher order ? Higher order functions are functions that take one or more functions as arguments, or return a function as their result. Like map(), reduce(),filter

map() returns a new array with each element being the result of the callback function. The original array is not modified.

filter() returns a new array with all elements that satisfy the condition specified by the callback function. The original array is not modified.

reduce() returns a single value, which is the final result of the reduction process. it will hold the computed value in the accumulator without changing the original value variable.

map(), filter() ,reduce() will take 3 parameter 1.function, 2.index, 3.array

.push() add element in array ending place

.pop() delete element in array ending place

.unshift() add element in array in starting place

.shift() delete element in array from starting position

.length length of array

.splice(index,delete,newElement) delete element and update element in array

splice() manupulate original array.

.slice(start, end) it take start and end parameter of index

slice() does not manupulate main array.

.sort(function(a, b){return a - b}) it take one function inside sort() and return ascending or descending

.at(index_num) find element based on index number

.toString() The toString() method returns an array as a comma separated string

["Banana", "Orange", "Apple", "Mango"]; = Banana,Orange,Apple,Mango

.join() join element of array in single string

["Banana", "Orange", "Apple", "Mango"]; = “Banana,Orange,Apple,Mango”

.reverse() reverse of array element

.concat(other_array) two or more array concat/add in single array

Different between map() and forEach() methode ?

.map() and .forEach() are both array methods that allow you to loop through an array, but they differ in what they return.

○ .map() returns a new array with the same length as the original array, where each element is the result of applying a callback function to the original element.

○ .forEach() does not return anything, but it simply executes a callback function on each element of the array.

Flattening an array in JavaScript is the process of converting a multi-dimensional array into a single one-dimensional array

---------- ---------- Object in js

What is object in javaScript

an object is a complex data type that allows you to store collections of data and more complex entities. Objects are fundamental to JavaScript and provide a way to group related data and functions (methods) together.

objectName.ObjkeyName print object items 1st methode

objectName["ObjkeyName"]; print object items 2nd methode

objectName[“objkeyName”]=”newitemsName” update or new item added

objectName.ObjkeyName=”newItemName” update or new item added

Delete object.objkeyName 1st method to delete obj items

Delete [“object.objkeyName”] 2nd method to delete obj items

Nested objects —---

const myObj =

{ name: "John",

age: 30,

myCars: {

car1: "Ford",

car2: "BMW",

car3: "Fiat"

}

}

Accessing nested object values

obj.innerObj.keyName or obj[“innerObj”]

[“innerObjKeyName”] constructor in js?

In JavaScript, a constructor is a special type of function used to create and initialize objects. Constructors allow you to create multiple instances of an object with the same structure and behavior.

```
function Person(name, age, isStudent) {
this.name = name;
this.age = age;
this.isStudent =
isStudent;
}
const alice = new Person("Alice", 30, false); //new object with same structure
const other = new Person("Alice", 30, false); //new object with same structure
```

Using the new Object() Syntax:

```
const person = new Object();
person.name = "Alice";
person.age = 30;
person.isStudent = false;
```

Using ES6 classes

```
class Person {
constructor(name, age, isStudent)
{ this.name = name;
this.age = age;
this.isStudent =
isStudent;
}
}
const alice = new Person("Alice", 30, false);
```

---

string—

In JavaScript, a string is a sequence of characters used to represent text. Strings can contain letters, numbers, symbols, and whitespace. They are one of the fundamental data types in JavaScript and are used frequently for various operations, such as storing and manipulating textual data.

Characteristics of Strings in JavaScript

1. Creation: Strings can be created using:
   - ○ Single quotes: 'Hello'
   - ○ Double quotes: "Hello"
   - ○ Backticks (template literals): `Hello`

String methods:

toUpperCase(): Converts to uppercase.

toLowerCase(): Converts to lowercase.

substring(): Extracts a part of the string.

indexOf(): Finds the position of a substring.

replace(): Replaces a substring with another string.

charAt(index no): char find using index

at(index no): char find using index no

slice(start, end): print char by index to index

first.concat(second): add two string into one

let str = "Hello";

console.log(str.length); // 5

let substr = str.substring(1, 4); // "ell"

let index = str.indexOf('e'); // 1

let newStr = str.replace('Hello', 'Hi'); // "Hi"

let upper = str.toUpperCase(); // "HELLO"

let lower = str.toLowerCase(); // "hello"

Template Literals

With backticks, you can create multi-line strings and include

variables. let name = "Alice";

let greeting = `Hello, ${name}!`; // "Hello, Alice!"

---------- ---------- — Dom-manipulation—

What is the DOM? The Document Object Model (DOM) is a programming interface

for web documents. It represents the page so that programs can change the

document structure, style, and content. The DOM represents the document as

nodes and objects; that way, programming languages can interact with the page.

OR

"The DOM (Document Object Model) is a structured tree representation of an HTML document. It allows JavaScript to interact with and manipulate webpage elements dynamically. Using DOM methods like getElementById() and querySelector(), we can change content, styles, and handle user interactions efficiently."

---------- ---------- — events in js

An event listener in JavaScript is a function that waits for a specific event occur on an element, such as a click or key press. You can add an event listener using the addEventListener method, which takes two main arguments: the event type (like "click") and the function to execute when the event occurs. This allows you to create interactive web pages.

preventDefault is a method in JavaScript that prevents the default action of an event from occurring. For example, when used in a form submission event, it stops the form from submitting and reloading the page

- ----------**********

event bubbling :- A propagation model where events are handled from then innermost element to the outermost
for event capturing example

```
document.querySelector("#child").addEventListener("click",function(event){
    console.log("child clicked1");
},false)
```

if i add false or its default false value its called is called event bubbling child to parant

event Capturing :- A propagation model where events are handled from the outermost element to the innermost

for event capturing example

```
document.querySelector("#child").addEventListener("click",function(event){
    console.log("child clicked1");
},true)
```

if i add true here so it capturing event parant to child

and use event.stopPropagation() for stop bubbling or capturing

****

Different between pure and impure function ?

Definition	A function that returns a value based on its input without modifying external variables.	A function that directly modifies a global variable or affects external state.
Takes Arguments?	✓ Yes	✗ No (or sometimes Yes, but still relies on global variables)
Returns a Value?	✓ Yes	✗ No (modifies state instead)
Modifies Global Variables?	✗ No	✓ Yes
Predictability	✓ Always returns the same output for the same input.	✗ Output can change due to external factors.
Side Effects?	✗ No	✓ Yes (changes global state)
Example	`function prior(x) { return x * 2; }`	`function unprior() { globalVar *= 2; }`

. Debouncing and **throttling ?**

Debouncing and **throttling** are techniques used in JavaScript to control how frequently a function is executed, especially for events like scrolling, and resizing.

Feature	**ouncing**	**Throttling**
Definition	Delays function execution until after a specified time since the last event.	Ensures function executes at most once per specified time interval.
Use Case	Search bars, window resizing, text input handling.	Scroll events, API rate-limiting, button click handling.
Execution	Executes only after the event stops firing for a delay period.	Executes once, then ignores repeated calls until the interval passes.
Example	A search bar that triggers an API request only after the user stops typing.	A scroll event listener that fires once every second instead of on every pixel scroll.

Example of Debouncing in JavaScript:

```
js
```

```
CopyEdit
function debounce(func, delay) {
    let timer;
    return function (...args) {
        clearTimeout(timer); // Clear the previous timer
        timer = setTimeout(() => func.apply(this, args), delay);
    };
}

// Example: Handling input search
function searchQuery(query) {
    console.log(`Searching for: ${query}`);
}

const debouncedSearch = debounce(searchQuery, 500); // Set delay to 500ms

// Simulate typing in a search box
debouncedSearch("H");
debouncedSearch("He");
debouncedSearch("Hel");
debouncedSearch("Hell");
debouncedSearch("Hello");

// Output (after 500ms): Searching for:
Hello
```

Example of Throttling in JavaScript:

```
js
CopyEdit
function throttle(func, interval) {
    let lastCall = 0;
    return function (...args) {
        let now = Date.now();
        if (now - lastCall >= interval) {
            lastCall = now;
            func.apply(this, args);


        }
    };
 }

// Example: Handling window scroll
function onScrollEvent() {
    console.log("Scroll event triggered", Date.now());
}

const throttledScroll = throttle(onScrollEvent, 1000); // Execute once every
1 second

// Simulate multiple scroll events
window.addEventListener("scroll", throttledScroll);
```

OOP Concept	Definition	Key Features	Example Use Case
Encapsulation	Restricts direct access to object data and methods.	- Uses private fields (#). - Uses getters and setters.	Bank account balance security.
Inheritance	Allows a child class to inherit properties and methods from a parent class.	- Uses `extends` for inheritance. - Uses `super()` to call parent constructor.	Student class extending Person class.
Polymorphism	Allows different classes to use the same method name with different implementations.	- Uses **method overriding**.	Animal sounds (Dog barks, Cat meows).
Abstraction	Hides complex details and enforces method implementation in subclasses.	- Cannot instantiate an abstract class. - Uses methods that must be overridden.	Car class extending Vehicle class.

1. **Encapsulation:-**

Example:

```
js
CopyEdit
class BankAccount {
    #balance; // Private field

    constructor(initialBalance) {
        this.#balance = initialBalance;
    }

    deposit(amount) {
if (amount > 0) {
       this.#balance += amount;
        }
    }

    getBalance() {
        return this.#balance; // Accessing private field
    }
}

const account = new BankAccount(1000);
account.deposit(500);
console.log(account.getBalance()); // Output: 1500
console.log(account.#balance); // Error: Private field cannot be accessed
```

2. **inheritance:-**

Example:

```
js
CopyEdit
class Person {
    constructor(name) {
        this.name = name;
    }

    greet() {
        return `Hello, my name is ${this.name}`;
    }
}

class Student extends Person {
    constructor(name, grade) {
        super(name); // Calls parent constructor
        this.grade = grade;
    }

    study() {
         return `${this.name} is studying for grade ${this.grade}`;
    }
}

const student = new Student("John", "A");
console.log(student.greet()); // Output: Hello, my name is John
console.log(student.study()); // Output: John is studying for grade A
```

3. **Polymorphism Example:**

```
class Animal {
    speak() {
        return "Some sound";
    }
}

class Dog extends Animal {
    speak() {
        return "Bark";
    }
}



class Cat extends Animal {
```

```
    speak() {
        return "Meow";
    }
}



let animals = [new Dog(), new Cat()];
animals.forEach(a => console.log(a.speak()));
// Output: Bark
// Output: Meow
```

4. Abstraction Example:

```
class Vehicle { constructor(make) {
        if (this.constructor === Vehicle) {
            throw new Error("Cannot instantiate abstract class");
        }
        this.make = make;
    }

         start() {
             throw new Error("Method 'start()' must be implemented");
    }
}

class Car extends Vehicle { start() {
        return `${this.make} car is starting...`;
    }
}

const car = new Car("Toyota");
console.log(car.start()); // Output: Toyota car is starting...

const vehicle = new Vehicle("Generic"); // Error: Cannot instantiate abstract
class
```

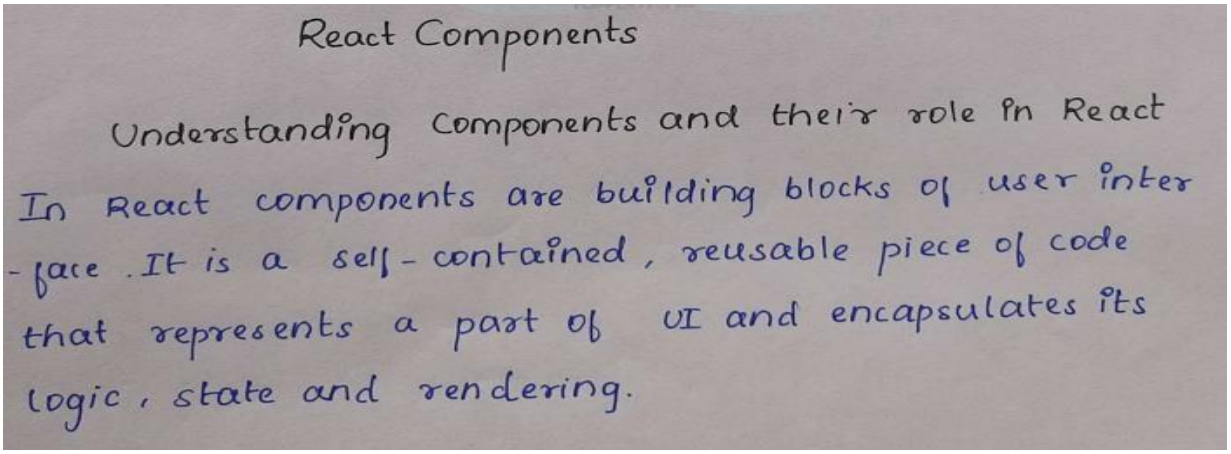
React js

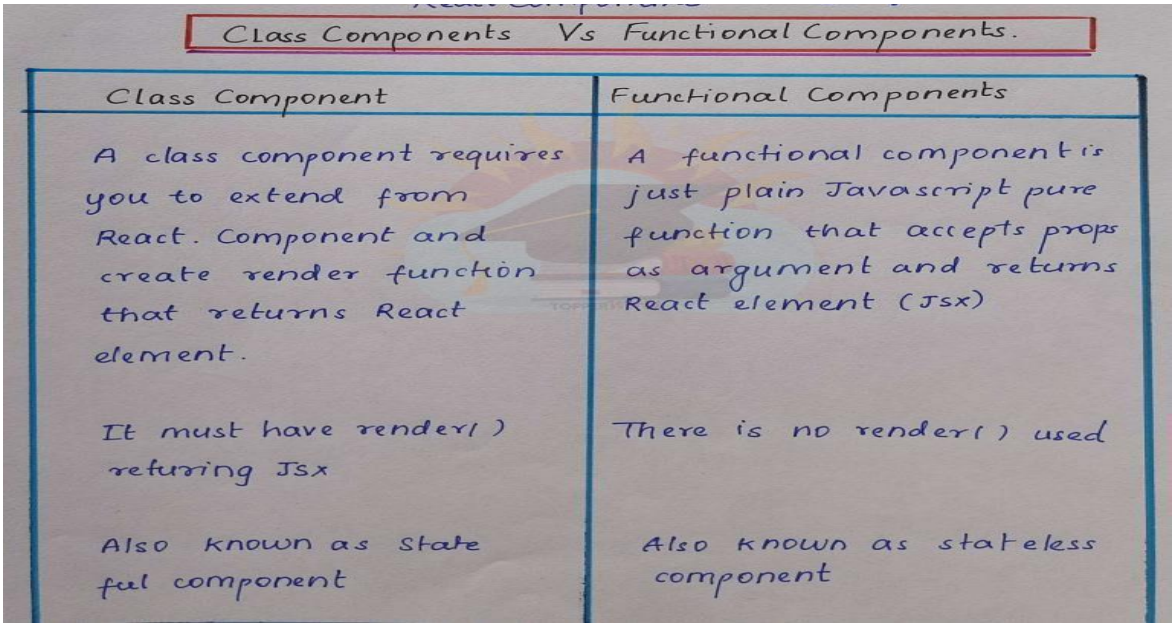
1. What is React.js?

React.js is an open-source JavaScript library developed by **Facebook** for building fast and interactive **user interfaces** (UIs), especially for **single-page applications (SPAs)**. It helps developers build reusable UI components and manage data effectively.

✵ Key Features of React.js:

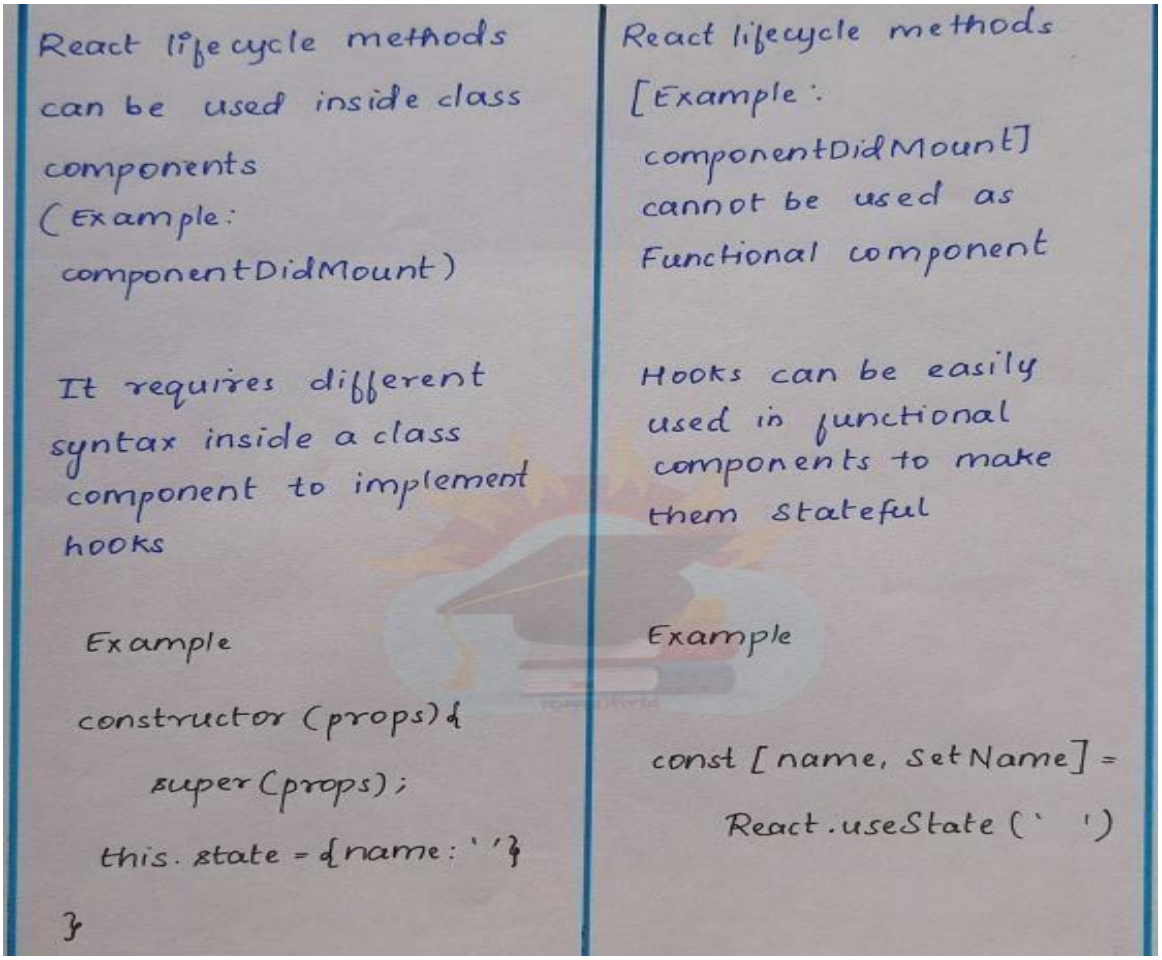
1. **Component-Based Architecture**
   a. UI is broken into independent, reusable components, making the code modular and easy to manage.
2. JSX (JavaScript XML)
   a. Allows writing HTML-like code inside JavaScript, making code more readable and concise.
3. Virtual DOM
   a. A lightweight copy of the real DOM that improves app performance by updating only changed elements.
4. Unidirectional Data Flow
   a. Data flows from parent to child components, making data management and debugging more predictable.
5. Hooks
   a. Introduced in React 16.8, Hooks allow using state and lifecycle methods in functional components.





Class Components Vs Functional Components.

Class Component	Functional Components
A class component requires you to extend from React.Component and create render function that returns React element.	A functional component is just plain Javascript pure function that accepts props as argument and returns React element (Jsx)
It must have render() returing Jsx	There is no render() used
Also known as State ful component	Also known as stateless component



React lifecycle methods can be used inside class components (Example: componentDidMount)

It requires different syntax inside a class component to implement hooks

Example

```
constructor (props){
    super(props);
this.state = {name: ''}
}
```

React lifecycle methods [Example: componentDidMount] cannot be used as Functional component

Hooks can be easily used in functional components to make them stateful

Example

```
const [name, setName] =
    React.useState('')
```

2. What is Significant of keys in react js

Keys are used to uniquely identify and differentiate between components in React. They help React identify which items have changed, added, or removed.

3. What is JSX ?

JSX is a React syntax extension that lets you write HTML-like code inside JavaScript, which gets compiled into `React.createElement()` calls to build the UI."

4. What is virtual DOM in React js ?

Virtual DOM is a lightweight copy of the real DOM. When state changes, React compares the new Virtual DOM with the old one and updates only the changed parts in the real DOM.

 How It Works:

1. **Initial Render:** React creates a virtual DOM tree that mirrors the actual DOM.
2. **State/Prop Change:** When data changes, React creates a **new virtual DOM tree**.
3. **Diffing:** React compares the new VDOM with the previous one using a **diffing algorithm**.
4. **Minimal Update:** React calculates the minimal number of changes and **updates only the changed parts** in the real DOM.

. What is Reconciliation in React?

Reconciliation is the process React uses to **compare the new Virtual DOM with the previous one** to determine the **most efficient way to update the real DOM**.

How It Works:

- When state/props change, React creates a new Virtual DOM.
- It then **diffs** it with the old one to detect changes.
- Only the changed elements are updated in the real DOM — this makes rendering fast and efficient.

This is the key to React's **high performance** and responsiveness.

. What is Strict Mode in React?

React Strict Mode is a tool for highlighting potential problems in your application during development.

It doesn't affect the UI but runs additional checks and warnings for:

- Unsafe lifecycle methods

- Unexpected side effects
- Deprecated APIs
- Double rendering in development to help catch issues

How to use:

```
<React.StrictMode>
  <App />
</React.StrictMode>
```

Note: It only works in development mode.

5. What is State?

State in React is a **built-in object** that stores **dynamic data** or **component-level data** that can change over time. It determines how a component behaves and what it renders on the UI.

6. What are props?

Props (short for **"properties"**) are **read-only inputs** passed from one component to another in React. They allow **data to flow from a parent component to a child component**, enabling reusability and dynamic rendering.

7. State vs props ?

Props are used to pass data from parent to child, while state is used to manage data within the component that may change over time."

8. What is hooks?

Hooks are special **functions** in React that let you **use state, lifecycle methods, and other React features** inside **functional components** — without writing a class.

G. What is a Lifecycle in React?

In React, the **component lifecycle** refers to the sequence of events that a component goes through **from creation to destruction**. It defines how a component is **initialized**, **updated**, and **removed** from the DOM.

React class components provide specific **lifecycle methods** that allow developers to execute code at different stages of this lifecycle.

The Main Lifecycle Phases:

1. **Mounting**
   The component is **created and inserted** into the DOM for the first time.
   → Example methods: `constructor()`, `render()`, `componentDidMount()`
2. Updating
   The component is **re-rendered** due to changes in **state** or **props**.
   → Example methods: `shouldComponentUpdate()`, `componentDidUpdate()`
3. Unmounting
   The component is **removed** from the DOM. Useful for cleanup tasks.
   → Example method: `componentWillUnmount()`
4. Error Handling (Optional)
   React catches **errors** in rendering or lifecycle methods of child components.
   → Example methods: `componentDidCatch()`, `getDerivedStateFromError()`

10. Hook	Description s Use Case
`useState()`	Adds **local state** to functional components.
`useEffect()`	Performs **side effects** like API calls, timers, or event listeners.
`useContext()`	Accesses **global data** (like themes, auth) without passing props manually.
`useRef()`	Stores **mutable values** or accesses **DOM elements**.
`useMemo()`	**Memoizes expensive calculations** to optimize performance.
`useCallback()`	**Memoizes functions** to avoid re-creating them unnecessarily.
`useReducer()`	Manages complex state logic (like Redux) in a functional way.
`useLayoutEffect()`	Like `useEffect`, but runs **synchronously after render** (used for layout reads).

11. 🔷 What is `useState()`?

`useState()` is a Hook that allows **functional components** to create and manage **local state**. When the state value changes, the component **re-renders** with the new value.

✅ Example:

```
const [count, setCount] = useState(0);
```

12. 🔷 What is `useEffect()`?

`useEffect()` is a Hook used to handle **side effects** in React components such as data fetching, DOM manipulation, or timers.

It runs **after the component renders**.

✅ Example:

```
useEffect(() => { fetchData();
}, []);
```

Dependency Array ([])	Behavior
No array	Runs after **every render**
Empty array `[]`	Runs **once** (like `componentDidMount`)
With values `[count]`	Runs only when **count changes** (like `componentDidUpdate`)

13. ◈ What is `useContext()`?

`useContext()` is a Hook that lets you **access context values** created using `React.createContext()` without using props.

It helps avoid **prop drilling** in deeply nested components.

■ Example:

```
const auth = useContext(AuthContext);
```

14. ◆ What is `useRef()`?

`useRef()` returns a **mutable object** that persists across renders.

It's often used to **reference DOM elements** or to **store values** that don't trigger re-renders when changed.

■ Example:

```
const inputRef = useRef(null);
```

15. ◈ What is `useMemo()`?

`useMemo()` is a Hook used to **memoize the result of an expensive function**, so it only recalculates when dependencies change. It improves performance by avoiding **unnecessary calculations**.

■ Example:

```
const result = useMemo(() => computeHeavyTask(data), [data]);
```

16. ◈ What is `useCallback()`?

`useCallback()` is used to **memoize functions**.

It prevents a function from being recreated unless its dependencies change, which helps **optimize performance**, especially in child components.

- Example:

```
const handleClick = useCallback(() => doSomething(), []);
```

17. ◈ What is `useReducer()`?

`useReducer()` is a Hook for managing **complex state logic** using a reducer function, similar to how Redux works.

It's useful when state depends on previous state.

- Example:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

18. ◈ What is `useLayoutEffect()`?

`useLayoutEffect()` is just like `useEffect`, but it runs **synchronously after DOM updates** and before the browser repaints the screen.

It's used for precise layout measurements or DOM manipulations.

- Example:

```
useLayoutEffect(() => {
  const height = ref.current.offsetHeight;
}, []);
```

19. What is Lazy loading ?

Lazy loading is a technique where components or modules are loaded only when they are actually needed, improving initial load times.

20. Explain the significance of the SuspenseList component in React. ?

SuspenseList is a component that allows developers to coordinate the loading of multiple components in a way that provides a better user experience

21. Controlled and Uncontrolled components ?

A controlled component's value is handled by React state, while an uncontrolled component's value is handled by the DOM using refs."

✅ What is a Controlled Component in React?

A **controlled component** is a form element (like `<input>`, `<textarea>`, `<select>`) whose value is controlled by React state.

React manages the data via `useState()` or `this.state`, and the value is updated using an onChange handler.

🔷 *Example (Controlled):*

```
const [name, setName] = useState('');

<input type="text" value={name} onChange={(e) => setName(e.target.value)}
/>
```

➞ React controls the value of the input field.

✅ What is an Uncontrolled Component in React?

An **uncontrolled component** manages its own state **internally using the DOM**, not React. You access the value using **Refs (useRef****)** instead of state.

🔷 *Example (Uncontrolled):*

```
const inputRef = useRef(null);

<input type="text" ref={inputRef} />
<button onClick={() => alert(inputRef.current.value)}>Submit</button>
```

➞ The DOM controls the input, not React.

22. What is a Higher-Order Component (HOC) in React?

A Higher-Order Component (HOC) is an advanced pattern in React where a function takes a component as input and returns a new enhanced component.

It's commonly used for **reusing component logic**, such as authentication, permissions, theming, logging, etc.

🧠 One-liner Definition:

"An HOC is a function that takes a component and returns a new component with additional functionality."

🔧 Syntax Example:

```
const withExtraInfo = (WrappedComponent) => { return
  function EnhancedComponent(props) {
    return (
      <div>
        <p>Extra Info Added</p>
        <WrappedComponent {...props} />
      </div>
    );
  };
```

```
};
```

✅ *Usage:*

```
const Hello = (props) => <h1>Hello, {props.name}</h1>; const
EnhancedHello = withExtraInfo(Hello);

// <EnhancedHello name="Manish" /> renders: Extra Info + Hello Manish
```

23. What is React Fragments ?

React Fragments allow developers to group multiple elements without adding an extra node to the DOM, helping to keep the structure clean.

24. What is Prop Drilling?

Prop drilling happens when you pass data from a **top-level (parent) component** to a deeply nested child component **through multiple intermediate components** — even if those middle components don't need the data.

25. How can be avoid Props driling ?

Prop drilling occurs when props are passed down through multiple levels of components. It can be avoided by using context or state management libraries like Redux.

26. ✅ What is a Router in React?

In React, a **router** refers to a library like **React Router** that allows you to **add navigation** and **multiple pages (routes)** in a single-page application (SPA) — without reloading the page.

React Router lets you map different URLs to different components, so you can build a **multi-view experience** inside one app.

React Router is a routing library that enables navigation between views in a React single-page application without reloading the browser

27. What is lifting the state up ?

Lifting state up is a pattern where the state of a child component is moved to its parent component, allowing multiple child components to share the same state.

Redux toolkit?

1. What is Redux?

Redux is a **predictable state management library** used for managing and centralizing the application state in JavaScript apps, especially React. It provides a **single source of truth (store)** where the entire app's state lives, and changes happen in a **predictable flow** using **actions** and **reducers**.

Core Principles:

- **Single source of truth** – one centralized store.
- **State is read-only** – only changed via actions.
- **Changes are made with pure functions** – called reducers.

2. What is `useSelector`?

`useSelector` is a React-Redux hook used to **select or extract specific data from the Redux store's state** in a functional component. It automatically **subscribes** the component to the store and re-renders it when the selected data changes.

Example:

```
const count = useSelector((state) => state.counter.value);
```

3. What is `useDispatch`?

`useDispatch` is a React-Redux hook that returns the `dispatch` method from the Redux store. It allows you to **send actions** from your component to the Redux store.

- Example:

```
const dispatch = useDispatch();
dispatch({ type: 'counter/increment' });
```

4. What is `createSlice`?

`createSlice` is a helper from Redux Toolkit that combines **action creators, action types, and reducers** into a single object called a "slice". It simplifies Redux logic.

- Example:

```
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },
  },
});

export const { increment } = counterSlice.actions;
export default counterSlice.reducer;
```

5. What are Reducers?

Reducers are **pure functions** in Redux that **take the current state and an action**, and **return a new state** based on the action type.

- Example:

```
const counterReducer = (state = { value: 0 }, action) => {
  switch (action.type) {
    case 'increment':
      return { ...state, value: state.value + 1 };
    default:
      return state;
  }
};
```

6. What is a Store?

The **Redux store** is a **central container** that holds the entire state of the application. It provides methods like `dispatch`, `getState`, and `subscribe`.

- Example using Redux Toolkit:

```
const store = configureStore({
  reducer: { counter: counterReducer },
});
```

7. What is `Provider`?

`Provider` is a component from `react-redux` that **wraps your entire app** and **makes the Redux store available** to all nested components through React context.

- Example:

```
<Provider store={store}>
  <App />
</Provider>
```

8. What is an Action?

An **action** is a plain JavaScript object that describes a **change or intention** to modify the state. It must include a `type` and can optionally include a `payload`.

- Example:

```
const action = {
  type: 'counter/increment',
  payload: 1
};
```

G. What is a Payload?

A **payload** is the **extra data** sent with an action to **inform the reducer** about what kind of change to make.

■ Example:

```
{
  type: 'counter/setValue',
  payload: 42
}
```

10. What is `configureStore`?

`configureStore` is a Redux Toolkit function that sets up the store with **sensible defaults**: like middleware, DevTools support, and combining multiple reducers.

■ Example:

```
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

const store = configureStore({
  reducer: { counter: counterReducer }
});
```

🔁 . What is Thunk Middleware in Redux?

Redux Thunk is a middleware that lets you write **async logic** (like API calls) inside Redux action creators.

Normally, Redux actions must return plain objects. But with Thunk, you can return a function that:

- Dispatches actions **before and after** an async task
- Accesses the store via `getState`

■ **Example:**

```
const fetchData = () => {

 return async (dispatch) => {

  dispatch({ type: 'FETCH_START' });
```

```
  const data = await fetch('/api/data').then(res => res.json());
  dispatch({ type: 'FETCH_SUCCESS', payload: data });
 };
};
```

Client-Side Rendering (CSR) and **Server-Side Rendering (SSR):**

🌐 What is Client-Side Rendering (CSR)?

Client-Side Rendering means that the browser downloads a blank HTML file and JavaScript bundle, then **renders the content dynamically** on the user's browser.

🔧 How it works:

1. Browser gets an empty HTML shell.
2. JavaScript loads and executes.
3. React (or any JS framework) renders the UI inside the browser.

✅ Pros:

- Faster navigation between routes (after initial load).
- Great for SPAs (Single Page Applications).

➕ Cons:

- Slower first-page load.
- Bad for SEO unless pre-rendered.

🖥️ What is Server-Side Rendering (SSR)?

Server-Side Rendering means that the content is rendered on the **server**, and the complete HTML is sent to the browser.

🔧 How it works:

1. Server generates HTML for each page request.
2. Browser receives and displays a fully rendered page.

3. React hydrates the page for interactivity.

✅ Pros:

- Faster initial page load.
- Better for SEO (content is immediately available to crawlers).

➕ Cons:

- Slower navigation (needs server roundtrip for each route).
- More server load.

Different between redux and context api?

“Context API is a React built-in feature for simple state sharing across components, ideal for light use cases like theme or auth info. Redux, on the other hand, is a full-fledged state management library suitable for large apps with complex global state, predictable updates, middleware support, and devtools for debugging. Essentially, Context solves prop drilling, Redux solves complex state management challenges.”

“Context triggers re-render for all consumers when its value changes, whereas Redux re-renders only the components that subscribe to the updated slice of state, making Redux more performant for large applications.

```jsx
// AuthContext.js
import { createContext, useState } from "react";

export const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState({ name: "A" });
  const [theme, setTheme] = useState("light");

  return (
    <AuthContext.Provider value={{ user, theme, setUser }}>
      {children}
    </AuthContext.Provider>
  );
}
```

Component 1 (Uses only user)

```jsx
// UserProfile.jsx
import { useContext } from "react";
import { AuthContext } from "./AuthContext";

export default function UserProfile() {
  const { user } = useContext(AuthContext);
  console.log("UserProfile rendered");

  return <h3>{user.name}</h3>;
}
```

Component 2 (Uses only theme)

```jsx
// Theme.jsx
import { useContext } from "react";
import { AuthContext } from "./AuthContext";

export default function Theme() {
  const { theme } = useContext(AuthContext);
  console.log("Theme rendered");

  return <p>{theme}</p>;
}
```

❌ What happens?

```js
setUser({ name: "B" });
```

Output:

```
UserProfile rendered
Theme rendered  ❌ (even though theme not changed)
```

```
<script>


    async function getData() {
        const res = await fetch('https://jsonplaceholder.typicode.com/posts')
        console.log(await res.json())
    }
    // getData()

    const res = fetch('https://jsonplaceholder.typicode.com/posts')
    res.then((value) => {
        console.log(value)
        return value.json()
    }).then((data) => {
        console.log(data)
    }).catch((err) => {
        console.log("errors", err)
    })

    const post = fetch('https://jsonplaceholder.typicode.com/posts').then((res) =>
res.json());
    const comment =
fetch('https://jsonplaceholder.typicode.com/posts/1/comments').then((res) => res.json());
    Promise.all([post, comment]).then((result) => {
        console.log('result', result)
    }).catch((error) => {
        console.log("error", error)
    })

</script>
```